LAPORAN PRAKTIKUM
PEMROGRAMAN 3 MINGGU KE-9

DOCKER VOLUME & NETWORK



Herdan Wahyu Mairendra Pangestu
NRP: 3325600064

Dosen Pengampu : Dian Septiani Santoso, M. Kom

PROGRAM STUDI D4 SAINS DATA TERAPAN
POLITEKNIK ELEKTRONIKA NEGERI SURABAYA

2026

HASIL PRAKTIKUM

Docker Volume

Latihan

Tujuan: membuktikan satu volume dapat dipakai bersama oleh dua container sekaligus, dan bahwa data di dalam volume tidak menghilang meski kontainernya dihapus.

1. Langkah 1: Membuat volume

Perintah ini menyediakan tempat penyimpanan bernama shared_data yang sepenuhnya dikelola Docker, sehingga umurnya tidak terikat pada satu container tertentu.

```
3325600064_Herdan:~$ docker volume create shared_data
shared_data
```

2. Langkah 2: Memastikan volume terbuat

Daftar volume ditampilkan guna memastikan shared_data benar-benar sudah terbentuk beserta driver yang menaunginya.

```
3325600064_Herdan:~$ docker volume ls
DRIVER    VOLUME NAME
local     07d11158f3cefd118a1990a2a3e345502e546f4775e61321f2b423b6232c84e7
local     shared_data
local     utspemrograman3_student-risk-data
```

3. Langkah 3: Menjalankan container ubuntu_a

-itd: container bekerja di latar belakang

--name ubuntu_a: penanda nama container

-v shared_data:/app: menyematkan volume shared_data ke folder /app di dalam container

sleep infinity: menjaga container tetap hidup karena ubuntu akan berhenti sendiri bila tidak ada program yang aktif

```
3325600064_Herdan:~$ docker run -itd --name ubuntu_a -v shared_data:/app ubuntu sleep infinity
6eca9bd82345a3f39be58a5f810b8efa768ed9052653979db8badfeb7b95d84a
```

4. Langkah 4: Menjalankan container ubuntu_b

Perintahnya identik dengan ubuntu_a, hanya berbeda nama. Karena memakai volume yang sama, isi /app di kedua container otomatis seragam.

```
3325600064_Herdan:~$ docker run -itd --name ubuntu_b -v shared_data:/app ubuntu sleep infinity
216eab6de7e6a2428b7846eb5aab9c115a2a0d88c9d4d787aac607c131314207
```

5. Langkah 5: Membuat file dari ubuntu_a

docker exec mengeksekusi perintah ke dalam container yang sedang aktif. Pada langkah ini file hello.txt ditulis ke /app sehingga data sebenarnya tersimpan di volume, bukan di lapisan

container.

```
3325600064_Herdan:~$ docker exec ubuntu_a sh -c 'echo hello docker > /app/hello.txt'
```

6. Langkah 6: Menampilkan isi file dari ubuntu_b

Isi hello.txt kemudian dibaca dari ubuntu_b; file yang ditulis ubuntu_a tadi diakses dari satu sumber penyimpanan yang sama.

```
3325600064_Herdan:~$ docker exec ubuntu_b cat /app/hello.txt
hello docker
```

7. Langkah 7: Menghapus kedua container

Flag -f memaksa penghapusan walau container sedang berjalan. Volume diperlakukan terpisah sehingga tidak ikut terbuang.

```
3325600064_Herdan:~$ docker rm -f ubuntu_a ubuntu_b
ubuntu_a
ubuntu_b
```

8. Langkah 8: Membuat container baru dengan volume yang sama

Container yang baru dibuat ditempelkan kembali ke shared_data lalu membaca isi /app/hello.txt. Isinya masih utuh — bukti bertahannya data volume setelah container semestanya dihapus.

```
3325600064_Herdan:~$ docker run --rm -v shared_data:/app ubuntu cat /app/hello.txt
hello docker
```

Tugas

Tujuan: membuat aplikasi Flask yang menyimpan data ke PostgreSQL, lalu membuktikan data tetap ada setelah docker compose down dan up.

1. Langkah 1: Membuat folder kerja

Semua file praktikum diwadahi folder tersendiri supaya rapi dan Compose mudah menemukan docker-compose.yml saat dijalankan.

```
3325600064_Herdan:~/tugas-volume$ mkdir tugas-volume

3325600064_Herdan:~/tugas-volume$ cd tugas-volume
```

2. Langkah 2: Membuat requirements.txt

Berisi pustaka yang dibutuhkan aplikasi: flask untuk melayani web server dan psycopg2-binary sebagai penghubung Python ke PostgreSQL.

```
3325600064_Herdan:~/tugas-volume$ cat > requirements.txt <<'EOF'

3325600064_Herdan:~/tugas-volume$ cat requirements.txt
flask
psycopg2-binary
```

3. Langkah 3: Membuat app.py

- get_conn(): membuka sesi koneksi ke database. host="db" memanfaatkan nama service db dari Compose; DNS internal Docker menerjemahkannya menjadi alamat server database

- init_db(): membuat tabel users saat aplikasi mulai berjalan, dengan pengulangan hingga 30 kali sebab Postgres menyertakan jeda beberapa detik sebelum siap

- /add/<name>: memasukkan nama pengguna baru ke tabel

- /users: menyajikan seluruh isi tabel

- /: beranda aplikasi

```
3325600064_Herdan:~/tugas-volume$ cat > app.py <<'EOF'

3325600064_Herdan:~/tugas-volume$ cat app.py
import os
import time
import psycopg2
from flask import Flask, jsonify

app = Flask(__name__)

def get_conn():
    return psycopg2.connect(
        host="db",
        dbname=os.environ["POSTGRES_DB"],
        user=os.environ["POSTGRES_USER"],
        password=os.environ["POSTGRES_PASSWORD"],
    )

def init_db():
    for _ in range(30):
        try:
            conn = get_conn()
            cur = conn.cursor()
            cur.execute("CREATE TABLE IF NOT EXISTS users (id SERIAL PRIMARY KEY, name TEXT)")
            conn.commit()
            conn.close()
            return
        except Exception:
            time.sleep(2)

@app.route("/")
def index():
    return "Flask + PostgreSQL jalan. Pakai /add/<nama> dan /users"

@app.route("/add/<name>")
def add_user(name):
    conn = get_conn()
    cur = conn.cursor()
    cur.execute("INSERT INTO users (name) VALUES (%s)", (name,))
    conn.commit()
    conn.close()
    return jsonify({"status": "ok", "added": name})

@app.route("/users")
def list_users():
    conn = get_conn()
    cur = conn.cursor()
    cur.execute("SELECT id, name FROM users ORDER BY id")
    rows = cur.fetchall()
    conn.close()
    return jsonify([{"id": r[0], "name": r[1]} for r in rows])

if __name__ == "__main__":
    init_db()
    app.run(host="0.0.0.0", port=5000)
```

4. Langkah 4: Membuat Dockerfile

FROM memilih citra dasar Python 3.12. WORKDIR menetapkan /app sebagai folder kerja container, COPY requirements.txt dan RUN pip install menyalin sekaligus memasang dependensi, COPY app.py menyalin kode aplikasi, dan CMD menjalankan aplikasi saat container dibangunkan.

```
3325600064_Herdan:~/tugas-volume$ cat > Dockerfile <<'EOF'

3325600064_Herdan:~/tugas-volume$ cat Dockerfile
FROM python:3.12
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY app.py .
CMD ["python", "app.py"]
```

5. Langkah 5: Membuat docker-compose.yml

Service db: memakai image postgres:16; bagian environment mengatur user, password, dan nama database; pemetaan pg_data:/var/lib/postgresql/data menaruh data database ke volume pg_data

Service web: dibangkitkan dari Dockerfile (build: .), membuka port 5000 ke komputer host, sementara depends_on memastikan db lebih dulu jalan

Bagian volumes di bawah: mendeklarasikan volume pg_data

```
3325600064_Herdan:~/tugas-volume$ cat > docker-compose.yml <<'EOF'

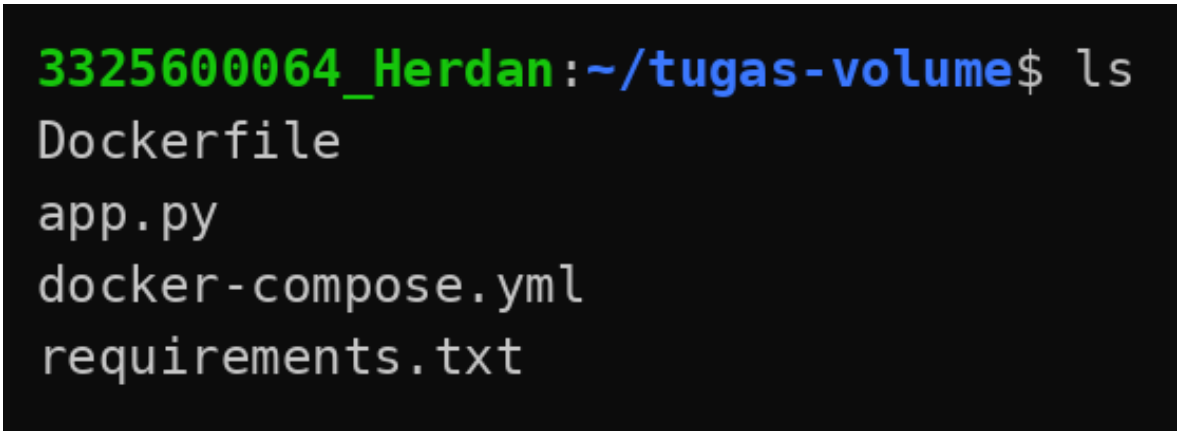
3325600064_Herdan:~/tugas-volume$ cat docker-compose.yml
services:
  db:
    image: postgres:16
    environment:
      POSTGRES_USER: admin
      POSTGRES_PASSWORD: secret
      POSTGRES_DB: praktikum
    volumes:
      - pg_data:/var/lib/postgresql/data

  web:
    build: .
    ports:
      - "5000:5000"
    environment:
      POSTGRES_USER: admin
      POSTGRES_PASSWORD: secret
      POSTGRES_DB: praktikum
    depends_on:
      - db

volumes:
  pg_data:
```

6. Langkah 6: Memeriksa file

Perintah ls memastikan keempat file project sudah lengkap terletak di folder.



7. Langkah 7: Menjalankan semua service

Image aplikasi Flask dibangun terlebih dahulu lalu service db dan web dihidupkan bersamaan di latar belakang (-d).

```
3325600064_Herdan:~/tugas-volume$ docker compose up -d --build
#1 [internal] load local bake definitions
#1 reading from stdin 564B done
#1 DONE 0.0s

#2 [internal] load build definition from Dockerfile
#2 transferring dockerfile: 168B done
#2 DONE 0.0s

#3 [internal] load metadata for docker.io/library/python:3.12
#3 DONE 0.0s

#4 [internal] load .dockerignore
#4 transferring context: 2B done
#4 DONE 0.0s

#5 [internal] load build context
#5 transferring context: 1.41kB done
#5 DONE 0.0s

#6 [1/5] FROM docker.io/library/python:3.12@sha256:0d0db5b17f11b1b7d1aafa74ceadaa64ba0944b267ff6fc22
008a197acd6b9ad
#6 resolve docker.io/library/python:3.12@sha256:0d0db5b17f11b1b7d1aafa74ceadaa64ba0944b267ff6fc22008
a197acd6b9ad 0.0s done
#6 DONE 0.0s

#7 [2/5] WORKDIR /app
#7 CACHED

#8 [3/5] COPY requirements.txt .
#8 CACHED

#9 [4/5] RUN pip install -r requirements.txt
#9 CACHED

#10 [5/5] COPY app.py .
#10 CACHED

#11 exporting to image
#11 exporting layers done
#11 exporting manifest sha256:f574a6e5dd856de7bb026ba7a2591102b9752733853cf0ff7b2e2fc39621c441
#11 exporting manifest sha256:f574a6e5dd856de7bb026ba7a2591102b9752733853cf0ff7b2e2fc39621c441 done
#11 exporting config sha256:f92575bf63384468f1b73928a97ad93756bdd0b92881d7b55384f1dc74cddcb4 done
#11 exporting attestation manifest sha256:632b10df890a3f66e828a8cff0640f58b747ed96039f6904550377b052
c3b376 done
#11 exporting manifest list sha256:5781267673020403280beb637de8b9fe811b8d6891b1b92307194b48492de542
done
#11 naming to docker.io/library/tugas-volume-web:latest done
#11 unpacking to docker.io/library/tugas-volume-web:latest done
#11 DONE 0.1s

#12 resolving provenance for metadata file
#12 DONE 0.0s

 Image tugas-volume-web Building
 Image tugas-volume-web Built
 Network tugas-volume_default Creating
 Volume tugas-volume_pg_data Creating
 Volume tugas-volume_pg_data Creating
 Network tugas-volume_default Creating
 Volume tugas-volume_pg_data Created
 Volume tugas-volume_pg_data Created
 Network tugas-volume_default Created
 Network tugas-volume_default Created
 Container tugas-volume-db-1 Creating
 Container tugas-volume-db-1 Created
 Container tugas-volume-web-1 Creating
 Container tugas-volume-web-1 Created
 Container tugas-volume-db-1 Starting
 Container tugas-volume-db-1 Started
 Container tugas-volume-web-1 Starting
 Container tugas-volume-web-1 Started
```

8. Langkah 8: Memeriksa status container

Kedua service tampak Up, menandakan web dan db sudah aktif dengan pengalamatan port sesuai konfigurasi.

```
3325600064_Herdan:~/tugas-volume$ docker compose ps
NAME                IMAGE               COMMAND                  SERVICE   CREATED          STATUS
       PORTS
tugas-volume-db-1    postgres:16         "docker-entrypoint.s…"   db        15 seconds ago   Up 15 se
conds   5432/tcp
tugas-volume-web-1   tugas-volume-web    "python app.py"          web       15 seconds ago   Up 15 se
conds   0.0.0.0:5000->5000/tcp, [::]:5000->5000/tcp
```

9. Langkah 9: Menambah data dari aplikasi

Permintaan dikirim ke endpoint /add untuk mencatat dua nama user ke dalam database.

```
3325600064_Herdan:~/tugas-volume$ curl http://localhost:5000/add/Ani
{"added":"Ani","status":"ok"}

  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed

  0      0   0      0   0      0      0      0                              0
100     30 100     30   0      0   3670      0                              0
100     30 100     30   0      0   3654      0                              0
100     30 100     30   0      0   3647      0                              0
3325600064_Herdan:~/tugas-volume$ curl http://localhost:5000/add/Budi
{"added":"Budi","status":"ok"}

  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed

  0      0   0      0   0      0      0      0                              0
100     31 100     31   0      0   4435      0                              0
100     31 100     31   0      0   4414      0                              0
100     31 100     31   0      0   4402      0                              0
```

10. Langkah 10: Menampilkan data sebelum container dihentikan

Endpoint /users membalas dua entri yang baru saja dimasukkan — saksi awal sebelum lingkungan dimatikan.

```
3325600064_Herdan:~/tugas-volume$ curl http://localhost:5000/users
[{"id":1,"name":"Ani"},{"id":2,"name":"Budi"}]

  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed

  0      0   0      0   0      0      0      0                              0
100     47 100     47   0      0   7251      0                              0
100     47 100     47   0      0   7215      0                              0
100     47 100     47   0      0   7197      0                              0
```

11. Langkah 11: Menghentikan container

docker compose down membuang container beserta networknya. Volume sengaja dibiarkan karena flag -v tidak digunakan.

```
3325600064_Herdan:~/tugas-volume$ docker compose down

 Container tugas-volume-web-1 Stopping
 Container tugas-volume-web-1 Stopped
 Container tugas-volume-web-1 Removing
 Container tugas-volume-web-1 Removed
 Container tugas-volume-db-1 Stopping
 Container tugas-volume-db-1 Stopped
 Container tugas-volume-db-1 Removing
 Container tugas-volume-db-1 Removed
 Network tugas-volume_default Removing
 Network tugas-volume_default Removed
```

12. Langkah 12: Memastikan volume masih ada

docker volume list menunjukkan volume masih tinggal meskipun seluruh container telah dihapus.

```
3325600064_Herdan:~/tugas-volume$ docker volume ls
DRIVER    VOLUME NAME
local     07d11158f3cefd118a1990a2a3e345502e546f4775e61321f2b423b6232c84e7
local     shared_data
local     tugas-volume_pg_data
local     utspemrograman3_student-risk-data
```

13. Langkah 13: Menjalankan ulang

Container baru diproduksi Compose sambil mengikat kembali volume lama yang isinya tidak berubah.

```
3325600064_Herdan:~/tugas-volume$ docker compose up -d

 Network tugas-volume_default Creating
 Network tugas-volume_default Creating
 Network tugas-volume_default Created
 Network tugas-volume_default Created
 Container tugas-volume-db-1 Creating
 Container tugas-volume-db-1 Created
 Container tugas-volume-web-1 Creating
 Container tugas-volume-web-1 Created
 Container tugas-volume-db-1 Starting
 Container tugas-volume-db-1 Started
 Container tugas-volume-web-1 Starting
 Container tugas-volume-web-1 Started
```

14. Langkah 14: Membuktikan data masih ada

/users masih mengembalikan data yang sama persis — data tidak ikut hilang saat stack di-down lalu di-up kembali.

```
3325600064_Herdan:~/tugas-volume$ curl http://localhost:5000/users
[{"id":1,"name":"Ani"},{"id":2,"name":"Budi"}]

  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed

  0      0   0      0   0      0      0      0                              0
100     47 100     47   0      0   6455      0                              0
100     47 100     47   0      0   6417      0                              0
100     47 100     47   0      0   6401      0                              0
```

Docker Network

Latihan

Tujuan: membuktikan container yang berada pada network berbeda tidak bisa saling berkomunikasi, dan hanya container yang tersambung ke kedua network (gateway) yang mampu menjangkau keduanya.

1. Langkah 1: Membuat dua network

frontend_net dan backend_net dibuat sebagai dua bridge terpisah; container yang berada di jaringan berbeda memang tidak saling terhubung secara bawaan.

```
3325600064_Herdan:~$ docker network create frontend_net
e2c0842674064e8b4c83cdd105820adee85d525ac3f4080f107acc9444b584c2
3325600064_Herdan:~$ docker network create backend_net
d8f5837a8bbcf35dfa5ed3f1b61303272ffc062ad5d21cb9d3a32eb2159cb40b
```

2. Langkah 2: Memastikan network terbuat

docker network ls menampilkan seluruh jaringan; dua jaringan baru berdriver bridge kini terlihat di daftar.

```
3325600064_Herdan:~$ docker network ls
NETWORK ID     NAME           DRIVER    SCOPE
d8f5837a8bbc   backend_net    bridge    local
aa3c36f0b798   bridge         bridge    local
e2c084267406   frontend_net   bridge    local
113eb648b442   host           host      local
940d00c0df26   none           null      local
```

3. Langkah 3: Menjalankan app_frontend

--network frontend_net menempatkan container langsung di jaringan frontend, sementara sleep infinity menjaganya tetap aktif.

```
3325600064_Herdan:~$ docker run -itd --name app_frontend --network frontend_net ubuntu sleep infinit
                     y
8ea6b5fbb76bdbd4429ef35ffcfc06b4779a9804923fcefa6372a1e92d1306bb
```

4. Langkah 4: Menjalankan app_backend

Cara membuatnya setara dengan app_frontend, hanya kali ini terikat pada backend_net.

```
3325600064_Herdan:~$ docker run -itd --name app_backend --network backend_net ubuntu sleep infinity
e434bb3e01bf8ca5cbcb828a27311155fca5db1574f83773d090f74ea5e1f3d5
```

5. Langkah 5: Menjalankan gateway

Pada awalnya gateway hanya melekat ke frontend_net; jaringan kedua akan disematkan pada langkah berikutnya.

```
3325600064_Herdan:~$ docker run -itd --name gateway --network frontend_net ubuntu sleep infinity
3112352f6fa75d705966ea7113a51a1b052da6a797292830fb156abb1b5d8f5d
```

6. Langkah 6: Menghubungkan gateway ke backend_net

docker network connect menambahkan koneksi kedua ke gateway. Sejak titik ini, gateway dapat menjangkau dua jaringan sekaligus.

```
3325600064_Herdan:~$ docker network connect backend_net gateway
```

7. Langkah 7: Memasang ping di ketiga container

Distribusi ubuntu minimal tidak membawa ping, jadi di setiap container terlebih dahulu dilakukan apt update lalu pemasangan iputils-ping.



8. Langkah 8: Tes ping dari gateway ke app_frontend

-c 3 membatasi uji ping sebanyak tiga kali. Container dipanggil dengan namanya, bukan alamat IP, karena DNS internal Docker.

```
3325600064_Herdan:~$ docker exec gateway ping -c 3 app_frontend
PING app_frontend (172.18.0.2) 56(84) bytes of data.
64 bytes from app_frontend.frontend_net (172.18.0.2): icmp_seq=1 ttl=64 time=0.057 ms
64 bytes from app_frontend.frontend_net (172.18.0.2): icmp_seq=2 ttl=64 time=0.056 ms
64 bytes from app_frontend.frontend_net (172.18.0.2): icmp_seq=3 ttl=64 time=0.059 ms

--- app_frontend ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2030ms
rtt min/avg/max/mdev = 0.056/0.057/0.059/0.001 ms
```

9. Langkah 9: Tes ping dari gateway ke app_backend

Karena gateway ikut menaungi backend_net, ping menuju app_backend juga berhasil.

```
3325600064_Herdan:~$ docker exec gateway ping -c 3 app_backend
PING app_backend (172.20.0.2) 56(84) bytes of data.
64 bytes from app_backend.backend_net (172.20.0.2): icmp_seq=1 ttl=64 time=0.053 ms
64 bytes from app_backend.backend_net (172.20.0.2): icmp_seq=2 ttl=64 time=0.053 ms
64 bytes from app_backend.backend_net (172.20.0.2): icmp_seq=3 ttl=64 time=0.055 ms

--- app_backend ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2053ms
rtt min/avg/max/mdev = 0.053/0.053/0.055/0.000 ms
```

10. Langkah 10: Tes ping dari app_frontend ke app_backend

Isolasi jaringan tampak di sini: keduanya menempati network berbeda sehingga nama app_backend gagal dikenali dan paket tidak pernah terkirim.

```
3325600064_Herdan:~$ docker exec app_frontend ping -c 3 app_backend

ping: app_backend: Name or service not known
```

11. Langkah 11: Melihat hostname dan IP semua container

Setiap container dimintai nama host beserta IP-nya. gateway memilah dua alamat IP karena melekat pada dua jaringan dalam waktu yang sama.

```
3325600064_Herdan:~$ docker exec app_frontend sh -c 'hostname; hostname -i'
8ea6b5fbb76b
172.18.0.2
3325600064_Herdan:~$ docker exec app_backend sh -c 'hostname; hostname -i'
e434bb3e01bf
172.20.0.2
3325600064_Herdan:~$ docker exec gateway sh -c 'hostname; hostname -i'
3112352f6fa7
172.18.0.3 172.20.0.3
```

12. Langkah 12: Membersihkan

Container dihapus lebih dahulu karena jaringan yang masih dipakai container tidak bisa dihapus; setelah itu frontend_net dan backend_net pun ditinggalkan.

```
3325600064_Herdan:~$ docker rm -f app_frontend app_backend gateway
app_frontend
app_backend
gateway
3325600064_Herdan:~$ docker network rm frontend_net backend_net
frontend_net
backend_net
```

Tugas

Tujuan: menjalankan dua service (web dan db) dalam satu network khusus app_network dengan Docker Compose, lalu membuktikan keduanya dapat berkomunikasi memakai nama service.

1. Langkah 1: Membuat folder kerja

Compose membaca docker-compose.yml dari folder aktif, sehingga folder khusus dibuat untuk menampung file tugas ini.

```
3325600064_Herdan:~/praktikum-network$ mkdir praktikum-network

3325600064_Herdan:~/praktikum-network$ cd praktikum-network
```

2. Langkah 2: Membuat docker-compose.yml

Service web: memakai image Nginx; port 80 container diteruskan ke port 8080 komputer melalui ports

Service db: memakai image MySQL 8; kata sandi root diatur melalui environment

Keduanya menunjuk ke network app_network, dan bagian networks di bawah mendeklarasikan jaringan bridge pribadi tersebut

```
3325600064_Herdan:~/praktikum-network$ cat > docker-compose.yml <<'EOF'

3325600064_Herdan:~/praktikum-network$ cat docker-compose.yml
services:
  web:
    image: nginx
    ports:
      - "8080:80"
    networks:
      - app_network

  db:
    image: mysql:8
    environment:
      MYSQL_ROOT_PASSWORD: 12345
    networks:
      - app_network

networks:
  app_network:
    driver: bridge
```

3. Langkah 3: Memeriksa file

Isi file compose diamati terlebih dahulu sebelum komponen dibangkitkan.

```
3325600064_Herdan:~/praktikum-network$ ls
docker-compose.yml
```

4. Langkah 4: Menjalankan service

Compose mengunduh image, membentuk network app_network, lalu menghidupkan web dan db di latar belakang.

```
3325600064_Herdan:~/praktikum-network$ docker compose up -d

 Network praktikum-network_app_network Creating
 Network praktikum-network_app_network Creating
 Network praktikum-network_app_network Created
 Network praktikum-network_app_network Created
 Container praktikum-network-web-1 Creating
 Container praktikum-network-db-1 Creating
 Container praktikum-network-web-1 Created
 Container praktikum-network-db-1 Created
 Container praktikum-network-web-1 Starting
 Container praktikum-network-db-1 Starting
 Container praktikum-network-web-1 Started
 Container praktikum-network-db-1 Started
```

5. Langkah 5: Memeriksa status container

Kedua service bertanda Up, menandakan web dan db berjalan dengan normal.

```
3325600064_Herdan:~/praktikum-network$ docker compose ps
NAME                      IMAGE     COMMAND                  SERVICE   CREATED        STATUS
          PORTS
praktikum-network-db-1    mysql:8   "docker-entrypoint.s…"   db        1 second ago   Up Less than a
 second   3306/tcp, 33060/tcp
praktikum-network-web-1   nginx     "/docker-entrypoint.…"   web       1 second ago   Up Less than a
 second   0.0.0.0:8080->80/tcp, [::]:8080->80/tcp
```

6. Langkah 6: Mengecek web berjalan

curl -I ke localhost:8080 dibalas 200 OK bertanda Server nginx, bermakna web sudah siap melayani.

```
3325600064_Herdan:~/praktikum-network$ curl -I http://localhost:8080
HTTP/1.1 200 OK
Server: nginx/1.31.8
Date: Mon, 26 Oct 2026 07:18:19 GMT
Content-Type: text/html
Content-Length: 896
Last-Modified: Wed, 21 Oct 2026 14:24:51 GMT
Connection: keep-alive
ETag: "6ad8cb33-380"
Accept-Ranges: bytes


  % Total    % Received % Xferd  Average Speed  Time    Time    Time   Current
                                 Dload  Upload  Total   Spent   Left   Speed

  0      0   0      0   0      0      0      0                              0
  0    896   0      0   0      0      0      0                              0
  0    896   0      0   0      0      0      0                              0
  0    896   0      0   0      0      0      0                              0
```

7. Langkah 7: Memeriksa network

Compose menambahkan nama folder sebagai prefix, sehingga jaringannya memakai nama praktikum-network_app_network.

```
3325600064_Herdan:~/praktikum-network$ docker network ls
NETWORK ID     NAME                            DRIVER    SCOPE
aa3c36f0b798   bridge                          bridge    local
113eb648b442   host                            host      local
940d00c0df26   none                            null      local
d2483e48821d   praktikum-network_app_network   bridge    local
```

8. Langkah 8: Memastikan web dan db di network yang sama

Hasil inspect memperlihatkan web dan db terhubung pada jaringan yang sama, masing-masing membawa alamat IP sendiri.

```
3325600064_Herdan:~/praktikum-network$ docker network inspect praktikum-network_app_network --format
                                       '{{range .Containers}}{{.Name}} -> {{.IPv4Address}}{{println
                                      }}{{end}}'
praktikum-network-web-1 -> 172.18.0.2/16
praktikum-network-db-1 -> 172.18.0.3/16
```

9. Langkah 9: Memasang ping di container web

Paket ping tidak tersedia pada image Nginx, jadi pemasangan iputils-ping didahulukan; docker compose exec web menjalankannya di dalam service web.

```
3325600064_Herdan:~/praktikum-network$ docker compose exec web bash -c 'apt-get update -qq && apt-ge
                                       t install -y iputils-ping'
Reading package lists...
Building dependency tree...
Reading state information...
The following additional packages will be installed:
  linux-sysctl-defaults
The following NEW packages will be installed:
  iputils-ping linux-sysctl-defaults
0 upgraded, 2 newly installed, 0 to remove and 5 not upgraded.
Need to get 57.0 kB of archives.
After this operation, 211 kB of additional disk space will be used.
Get:1 http://deb.debian.org/debian trixie/main amd64 iputils-ping amd64 3:20240905-3 [51.2 kB]
Get:2 http://deb.debian.org/debian trixie/main amd64 linux-sysctl-defaults all 4.12.1 [5724 B]
Fetched 57.0 kB in 0s (250 kB/s)
Selecting previously unselected package iputils-ping.
(Reading database ...
(Reading database ... 5%
(Reading database ... 10%
(Reading database ... 15%
(Reading database ... 20%
(Reading database ... 25%
(Reading database ... 30%
(Reading database ... 35%
(Reading database ... 40%
(Reading database ... 45%
(Reading database ... 50%
(Reading database ... 55%
(Reading database ... 60%
(Reading database ... 65%
(Reading database ... 70%
(Reading database ... 75%
(Reading database ... 80%
(Reading database ... 85%
(Reading database ... 90%
(Reading database ... 95%
(Reading database ... 100%
(Reading database ... 6724 files and directories currently installed.)
Preparing to unpack .../iputils-ping_3%3a20240905-3_amd64.deb ...
Unpacking iputils-ping (3:20240905-3) ...
Selecting previously unselected package linux-sysctl-defaults.
Preparing to unpack .../linux-sysctl-defaults_4.12.1_all.deb ...
Unpacking linux-sysctl-defaults (4.12.1) ...
Setting up linux-sysctl-defaults (4.12.1) ...
Setting up iputils-ping (3:20240905-3) ...

debconf: unable to initialize frontend: Dialog
debconf: (TERM is not set, so the dialog frontend is not usable.)
debconf: falling back to frontend: Readline
debconf: unable to initialize frontend: Readline
debconf: (Can't locate Term/ReadLine.pm in @INC (you may need to install the Term::ReadLine module)
(@INC entries checked: /etc/perl /usr/local/lib/x86_64-linux-gnu/perl/5.40.1 /usr/local/share/perl/5
.40.1 /usr/lib/x86_64-linux-gnu/perl5/5.40 /usr/share/perl5 /usr/lib/x86_64-linux-gnu/perl-base /usr
/lib/x86_64-linux-gnu/perl/5.40 /usr/share/perl/5.40 /usr/local/lib/site_perl) at /usr/share/perl5/D
ebconf/FrontEnd/Readline.pm line 8, <STDIN> line 2.)
debconf: falling back to frontend: Teletype
debconf: unable to initialize frontend: Teletype
debconf: (This frontend requires a controlling tty.)
debconf: falling back to frontend: Noninteractive
```

10. Langkah 10: Ping db dari web

web mengirim ping langsung ke db hanya dengan nama service, tanpa menyebut IP sama sekali.

```
3325600064_Herdan:~/praktikum-network$ docker compose exec web ping -c 3 db
PING db (172.18.0.3) 56(84) bytes of data.
64 bytes from praktikum-network-db-1.praktikum-network_app_network (172.18.0.3): icmp_seq=1 ttl=64 t
ime=0.072 ms
64 bytes from praktikum-network-db-1.praktikum-network_app_network (172.18.0.3): icmp_seq=2 ttl=64 t
ime=0.070 ms
64 bytes from praktikum-network-db-1.praktikum-network_app_network (172.18.0.3): icmp_seq=3 ttl=64 t
ime=0.051 ms

--- db ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2045ms
rtt min/avg/max/mdev = 0.051/0.064/0.072/0.009 ms
```

11. Langkah 11: Membersihkan

docker compose down memberhentikan serta membersihkan container dan jaringan app_network.

```
3325600064_Herdan:~/praktikum-network$ docker compose down

 Container praktikum-network-db-1 Stopping
 Container praktikum-network-web-1 Stopping
 Container praktikum-network-web-1 Stopped
 Container praktikum-network-web-1 Removing
 Container praktikum-network-web-1 Removed
 Container praktikum-network-db-1 Stopped
 Container praktikum-network-db-1 Removing
 Container praktikum-network-db-1 Removed
 Network praktikum-network_app_network Removing
 Network praktikum-network_app_network Removed
```